StaC - Stack Transfer and Consume

---

1. Introduction

1.1 Design Philosophy

StaC is a C-like language built on one premise: **everything is a stack**. There are no arrays are no arrays, only stacks of assigned values. Assignment is replaced by directional **transfer operators** that move values between stacks, always consuming the source. Reading a variable without consuming it is not possible through ordinary expressions and duplications must be explicit.

This is intentionally inconvenient, but does allow programing tasks to be completed. The goal is think a little differently about data.

The central tensions of StaC is the following:

- **Move vs Copy** - there is no implicit copy. `dup()` is the only way to duplicate a value, and must be called.
- **Stack Depth as Memory** - A variable's history lives below its top. Forgetting to drain a stack is a semantic error.
- **Transfer Direction is Explicit** - You cannot accidentally assign. Every transfer is a visible, directional arrow in the from source to target.
- **Truthiness is structural** - There is no boolean type. A stack with values are true. An empty stack is false.

1.2 Influences

C and C++ are the primary influences for the syntax. StaC adopts C's type system, control flow style, function declaration, and block structure. it also adopts C's type philosophy - type mismatches are permitted, but its on the programmer to resolve these.

SNOBOL Influenced the the idea of one global container, but instead of strings, we have stacks. Of course these stacks are typed, but thats mostly for readability and printability. This also influenced the function return model. In SNOBOL, functions return values by assigning to the functions's own name. StaC adopts this directly: inside a function body, the function name is a live stack variable. Pushing values onto it accumulates the return value, and `return` yields whatever was pushed.

---

2. EBNF Grammar

```
program           = {function_def | declaration} ;

(*decalarations*)
declaration       = type , identifier , [ ( "<=" | "<==" ) , expression ] , ";" ;
type              = "int" | "char" ;

(*functions*)
function_def      = type , identifier , "(" , [ param_list ] , ")" , "{" , { statement
} , "}" ;
param_list        = param , { "," , param } ;
param             = type , identifier ;

(*statements*)
statement         = declaration
                  | transfer_stmt
                  | dump_stmt
                  | top_swap_stmt
                  | full_swap_stmt
                  | if_stmt
                  | while_stmt
                  | for_stmt
                  | return_stmt
                  | builtin_call_stmt ;

transfer_stmt     = sink , "<=" , expression , ";" ;
dump_stmt         = sink , { "<== , expression } , ";" ;
top_swap_stmt     = identifier , "<=>" , identifier , ";" ;
full_swap_stmt    = idenfifier , "<==>" , identifier, ";" ;

sink              = identifier | "print" ;

(*control flow*)
if_stmt           = "if" , "(" , condition , ")" ,
                    "{" , { statement } , "}" ,
                    [ "else" , "{" , { statement } , "} ] ;

while_stmt        = "while" , "(" , condition , ")" ,
                    "{" , { statement } , "}" ;

for_stmt          = "for" , "(" , for_init , ";" , condition , ";" , for_step ")" ,
                    "{" , { statement } , "}" ;
for_init          = declaration ;
for_step          = identifier, "<=", expression ;

(*conditions*)
condition         = expression , comp_op, expression
                  | "!" , identifier
                  | identifier
                  | "empty" , "(" , identifier, ")" ;
```

```
                  | "!" , "empty" , "(" , identifier , ")" ;

comp_op           = "<" | ">" | "=" | "!=" | "!<" | "!>" ;

(* expressions *)
expression        = term, { ( "+" | "-" ) , term } ;
term              = factor , { ( "*" | "/" | "%" ), factor} ;
factor            = identifier
                  | literal
                  | function_call
                  | builtin_expr
                  | "(" , expression, ")" ;

function_call     = identifier , "(" , [ arg_list] , ")" ;
arg_list          = identifier , { "," , identifier } ;

builtin_expr      = "randInt" , "(" , expression , "," , expression , ")"
                  | "scanInt" , "(" , ")"
                  | "scan" , "(" , ")"
                  | "depth" , "(" , identifier , ")" ;

builtin_call_stmt = ( "dup" | "pop" ) , "(" , identifier , ")" , ";" ;

(* literals *)
literal           = int_literal | char_literal | string_literal ;
int_literal       = digit , { digit } ;
char_literal      = "'" , character , "'" ;
string_literal    = '"' , { character } '"' ;

(* primitives *)
identifier        = letter, { letter | digit | "_" } ;
letter            = "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h" | "i" | "j" | "k"
                  | "l" | "m" | "n" | "o" | "p" | "q" | "r" | "s" | "t" | "u" | "v"
                  | "w" | "x" | "y" | "z" | "A" | "B" | "C" | "D" | "E" | "F" | "G"
                  | "H" | "I" | "J" | "K" | "L" | "M" | "N" | "O" | "P" | "Q" | "R"
                  | "S" | "T" | "U" | "V" | "W" | "X" | "Y" | "Z" ;
digit             = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;
character         = letter | digit | " " | "!" | "#" | "$" | "%" | "&" | "'" | "("
                  | ")" | "*" | "+" | "," | "-" | "." | "/" | ":" | ";" | "<" | "="
                  | ">" | "?" | "@" | "[" | "]" | "^" | "_" | "`" | "{" | "|" | "}"
                  | "~" | "\n" | "\t" | "\r" | "\\" | "\"" ;
```

---

3. Semantic Behavior

3.1 Variable Binding

Every variable in StaC is a **stack** - an ordered collection of typed values with the most recently pushed value on top. Declaring a variable creates an empty stack of the declared type. Variables are bound to a name within their enclosing scope and do not exist outside of it.

```
int x; //x is an empty int stack, bound in current scope
int y <= 10; //y is an int stack with 10 on top
```

Scope follows C rules. Variables declared inside a block `{ }` are not accessible outside it. Function parameters are scoped to the function body. When a scope exits, all stacks bound within it are destroyed, including any values they contain.

Type is fixed at declaration and applies to every value in the stack. StaC uses C-level type enforcement - mismatched transfers are permitted, but undefined behavior may occur if not careful.

Stack Depth is unbounded. Values accumulate below the top as transfers push new values. Forgetting to drain a stack does not produce an error.

Truthiness is an intrinsic property of every stack, not a value type. A stack with one or more values is **truthy**. An empty stack is **falsy**. There is no boolean type and no boolean literal. Comparisons are not expressions and cannot produce storable values - they exist only as conditions in control flow.

3.2 Transfer Operators

`<=` Single Transfer

Evaluates right-hand expression, consuming the top of every variable referenced, and pushes the result onto the left-hand side. Expressions are handled like C and strings will evaluate to the right-most character (likely \n).

```
int x <= 5; //x: [5]
int y <= x; //y: [5], x: [] (consumed)
y <= y + 1; //y consumed, 5 + 1 = 6 gets pushed

char a <= "hello world"; //a: [d]. only right most character gets consumed
```

`<==` Dump Transfer

Repeatedly applies `<=` until the shortest operand is exhausted. Remaining values in longer operands are left unconsumed.

```
a <== b + c; //takes the top of b and c, adds them, and pushes to a repeatedly until b
or c is empty
```

`<=>` Top Swap

Swaps the top values of the two stacks. If one is empty, the empty stack gets moved to the other stack, essentially degrading to a move.

```
int a <== 1 <= 2;
int b <== 2 <= 3;
a <=> b; // a: [2,2], b:[1,3] after swap
```

`<==>` Full Swap

Swaps the entire contents of two stacks. All values, depth, and order are exchanged.

```
int a <== 1 <= 2;
int b <== 2 <= 3;
a <==> b; // a: [2,3], b:[1,2] after swap
```

3.3 Control Structures

`if` / `else`

Evaluates a condition. Conditions never consume values. A bare identifier is a truthy check - true if stack is non-empty.

```
if(x){
    //x is non-empty
}

if(y = 5){
    //top of y is 5
}
else {
    y <= y + 1;
}
```

`while`

Repeatedly evaluates a condition and executes the body while it holds true. Using a `while(x)` is the best way to drain a stack.

```
while(x){
    pop(x);
}
```

`for`

A counted loop with an initialization declaration, a non-consuming condition, and a transfer-based step.

```
for(int i <= 0; i < 10; i <= i + 1){
    dup(i);
    print <= i;
}
```

Type Keywords

Keyword	Meaning
`int`	Integer stack
`char`	Character stack

Control Keywords

Keyword	Meaning
`if`	Conditional branch
`else`	Alternate branch
`while`	Condition-checked loop
`for`	Counted loop
`return`	Exit function, yield function name stack as return value

Transfer Operators

Operator	Meaning
`<=`	Move top of right into left
`<==`	Repeated transfer until shortest operand exhausted
`<=>`	Swap tops of two stacks
`<==>`	Swap entire stacks

Comparison Operators (non-consuming, conditions only)

Operator	Meaning
`<`	Less than
`>`	Greater than
`=`	Equal
`!=`	Not equal
`!<`	Greater than or equal
`!>`	Less than or equal

Form	Meaning
`x`	True if stack `x` is non-empty
`!x`	True if stack `x` is empty
`empty(x)`	Alias for `!x`
`!empty(x)`	Alias for `x`

Built-in Functions

Function	Meaning
`dup(x)`	Duplicate top of stack `x`. The only way to copy a value.
`pop(x)`	Discard top of stack `x`.
`depth(x)`	Returns the number of values in stack `x` as an `int`. Non-consuming.
`empty(x)`	True if stack `x` has no values. Alias for `!x` in conditions.
`print`	Transferring to `print` outputs the value and discards it.
`scan()`	Reads a line of user input. Returns a `char` stack, first character on top.
`scanInt()`	Reads an integer from user input. Returns a single `int`.
`randInt(min, max)`	Returns a random `int` between `min` and `max` inclusive. Both arguments consumed.

---

4. Code Examples

4.1 Hello World

```
int main(){
    char output <== "Hello World!\n";
    print <== output;
}
```

OR

```
int main(){
print <== "\n!dlroW olleH";
}
```

4.3 Prompt Counter

```
int main(){
    char prompt_start <== "Enter start: ";
    char prompt_end   <== "Enter end: ";

    print <== prompt_start;
    int start <= scanInt();

    print <== prompt_end;
    int end <= scanInt();

    for (int i <= start; i !> end; i <= i + 1){
        dup(i);
        print <= i;

        dup(i);
        rem <= i % 2;

        if(rem = 0){
            char even <= " even\n";
            print <== even;
        }
        else{
            char odd <= " odd\n";
            print <== odd;
        }
    }
}
```

Min/Max Finder

```
// finds the minimum value
// nums is the number stack
// returns a stack with the minumum value on top
int findMin(int nums){
    //assume value on top is the minimum
    findMin <= nums;

    while(nums){
        //push as return value if the top on nums is less than known min
        if(nums < findMin){
            findMin <= nums;
        }
        //pop top of nums, a new minimum is not found
        else{
            pop(nums);
        }
    }

    return;
}

//finds the maxiumum value
// nums is the number stack
// returns a stack with the maximum value on the top
int findMax(int nums){
    //assume value on top is the maximum
    findMax <= nums;

    while(nums){
        //push as return value if the top on nums is less than known min
        if(nums > findMax){
            findMax <= nums;
        }
        //pop top of nums, a new minimum is not found
        else{
            pop(nums);
        }
    }

    return;
}

int main(){
    int n;
    int nums;

    char output <== "How many numbers do you want to add?\n";
    print <== output;
```

```
    n <= scanInt();

    char reverse <== " numbers.\n"
    dup(n);
    reverse <= n;
    output <== "Please insert " <== reverse;

    print <== output;
    for(i <= 0; i < n; i <= i + 1){
        nums <= scanInt();
    }

    int min <= findMin(nums);
    int max <= findMax(nums);

    reverse <== '\n';
    reverse <== min;
    output <== "Max: " <== reverse;
    print <==  output;
    reverse <== '\n';
    reverse <== max;
    output <== "Min: " <== reverse;
    print <== output;
}
```